MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

A TWO-TIER COGNITIVE ARCHITECTURE FOR AUTOMATED THREAT DETECTION AND RESPONSE USING AGENTIC AI

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

A TWO-TIER COGNITIVE ARCHITECTURE FOR AUTOMATED THREAT DETECTION AND RESPONSE USING AGENTIC AI

by

Nguyen Duc Binh

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:
Dr. Bui Van Hieu
Dr. Dang Van Hieu

Abstract

In the contemporary cybersecurity landscape, Security Operations Centers (SOCs) are consistently overwhelmed by the sheer volume of network alerts, leading to severe alert fatigue and the potential oversight of critical Advanced Persistent Threats (APTs). To address the inherent limitations of traditional, static rule-based systems and the latency bottlenecks of processing raw logs directly through Large Language Models (LLMs), this thesis introduces SENTINEL: A Two-Tier Cognitive Architecture for Automated Threat Detection and Response. The proposed framework operates by deploying Welford's online statistical algorithm at the Transport Tier (Tier-1) to filter benign traffic at wire speed with minimal computational overhead. Subsequently, the Cognitive Agent Tier (Tier-2) leverages a LangGraph-driven AI Agent equipped with a Dual Retrieval-Augmented Generation (Dual-RAG) mechanism—integrating MITRE ATT&CK and NIST guidelines—to perform deep, contextual threat reasoning and mitigation.

Evaluated on the CSE-CIC-IDS2018 and DAPT2020 datasets, SENTINEL demonstrates substantial improvements in both detection accuracy and processing efficiency. Furthermore, the system incorporates robust AI security guardrails, including Delimited Data Encapsulation and HMAC log chaining, achieving near-perfect mitigation against known adversarial AI manipulation attacks and ensuring forensic data integrity. The findings underscore the efficacy of combining deterministic statistical filters with explainable Agentic AI, offering a highly scalable and secure automation solution for modern network defense.

Acknowledgments

I would like to express my deepest and most sincere gratitude to my supervisors, Dr. Bui Van Hieu and Dr. Dang Van Hieu. Their precise academic direction, sharp professional feedback, and continuous guidance throughout this research have served as the cornerstone for the successful completion of this thesis.

I am also profoundly grateful to all the faculty members and staff at FPT University, particularly the Master of Software Engineering (MSE) program, for equipping me with invaluable academic knowledge, fostering a highly professional and modern research environment, and providing excellent resources during my master's studies.

Furthermore, I would like to extend my appreciation to the management and my colleagues at FPT Software for their support, understanding, and valuable professional discussions, which greatly helped me balance my professional duties and academic pursuits.

Finally and most importantly, I dedicate a special acknowledgment to my loving family, especially my wife and young child. Their silent sacrifices, patience, understanding, and unwavering moral support during my intensive experimental runs provided the ultimate strength and motivation for me throughout this master's journey.

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Description
AI	Artificial Intelligence
APT	Advanced Persistent Threat
C2	Command and Control
CIDR	Classless Inter-Domain Routing
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CVE	Common Vulnerabilities and Exposures
CTI	Cyber Threat Intelligence
DAPT	Dynamic/Delayed Advanced Persistent Threat
DDoS	Distributed Denial of Service
DL	Deep Learning
DoS	Denial of Service
EDR	Endpoint Detection and Response
FAISS	Facebook AI Similarity Search
FSM	Finite State Machine
GenAI	Generative Artificial Intelligence
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HMAC	Hash-based Message Authentication Code
IDS	Intrusion Detection System
IP	Internet Protocol
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAS	Multi-Agent System
ML	Machine Learning
NGFW	Next-Generation Firewall
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RBA	Runbook Automation
RRF	Reciprocal Rank Fusion
SCA	Software Composition Analysis
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SOC	Security Operations Center
SVM	Support Vector Machine
TDIR	Threat Detection and Incident Response
TTP	Tactics, Techniques, and Procedures
TTL	Time to Live
VRAM	Video Random Access Memory
WAF	Web Application Firewall
YAML	YAML Ain't Markup Language

Contents

Chapter 1

Introduction

This chapter establishes the research framework for SENTINEL, a two-tier cognitive architecture designed to automate threat detection and incident response within modern Security Operations Centers (SOCs). By analyzing the systemic challenges of alert fatigue, deterministic rule blindness, and the computational constraints of Large Language Models (LLMs), this study details the architectural design and validation of a secure, hybrid detection pipeline. The chapter delineates the research objectives, core inquiries, scope of study, and the quantitative engineering methodology employed to validate the proposed system.

1.1 Background and Motivation

Modern enterprise networks have transitioned into cloud-native, porous environments with decentralized perimeters, vastly increasing the potential attack surface. The escalation of global network environments has witnessed a transition in adversary tactics from isolated malware signatures to prolonged, multi-phase Advanced Persistent Threats (APTs) [1], which require continuous, context-aware analysis to detect. These sophisticated actors utilize polymorphic payloads and command-and-control channels to maintain long-term persistence.

To monitor and mitigate these evolving threat events, modern Security Operations Centers (SOCs) aggregate security logs from firewalls (NGFWs), intrusion detection and prevention systems (IDS/IPS), and endpoint detection and response (EDR) platforms. Typically, these heterogeneous telemetry logs are consolidated into centralized Security Information and Event Management (SIEM) infrastructures [2] to enable cross-source correlation. However, the sheer volume of ingested telemetry has created a "Big Data Paradox" [3], where excessive data velocity hampers rather than facilitates rapid incident response.

This data overflow directly leads to a severe alert fatigue crisis. Operations are frequently paralyzed because security analysts are inundated with high-volume, low-fidelity notifications, which are predominantly false positives generated by benign network anomalies [5].

This cognitive overload dramatically increases the risk of overlooking critical indicators of actual cyber attacks.

Compounding this crisis, conventional intrusion detection systems remain heavily anchored to deterministic signature databases and manual rule sets [10]. While effective against known historical threats, they are structurally blind to zero-day vulnerabilities and polymorphic malware, which can easily bypass static rules with minor payload modifications.

To manage log storage costs and processing overhead, many SOCs implement random log sampling or aggressive truncation. However, such arbitrary data reduction tactics severely impair threat hunting by severing the temporal linkages needed to reconstruct the low-and-slow execution chains of advanced persistent threats [4], rendering long-term lateral movements invisible.

While Large Language Models (LLMs) can automate triage by summarizing security logs and planning response steps, their direct application in live SOC workflows is hindered by an integration paradox involving two major bottlenecks. First, the inference latency of multi-billion parameter LLMs is computationally prohibitive for raw, high-velocity network packet logs due to the self-attention mechanisms of the Transformer architecture. Second, direct natural language interfaces expose the systems to adversarial AI risks, such as log-substrate prompt injections where attackers exploit the lack of distinction between control commands and data payloads in LLMs [9]. Malicious payloads embedded in log files (e.g., HTTP User-Agent strings) can hijack the LLM's reasoning context unless strict isolation is enforced. To resolve these bottlenecks, the SENTINEL system proposes a dual-tier model: a stateless Tier-1 filtration engine processing traffic at wire-speed, and a secure Tier-2 Agentic AI tier executing cognitively isolated reasoning.

1.2 Research Objectives

The general objective of this research is to design, implement, and quantitatively evaluate the SENTINEL two-tier cognitive architecture, combining low-latency statistical anomaly filtration with a secure, context-aware Agentic AI reasoning workflow.

Specifically, the research focuses on three key objectives. The first objective is to implement Welford's online statistical algorithm for $O(1)$ rolling calculation of variance and Z-Scores to filter benign network traffic at wire-speed. The second objective is to design a stateful autonomous agent using LangGraph and local quantized LLMs, integrated with input-output guardrails, to securely analyze anomalies and mitigate adversarial attack vectors targeting LLM-integrated systems. Finally, the third objective is to build a dual RAG system integrating MITRE ATT&CK and NIST SP 800-61r2 playbooks to ground agent decisions and minimize hallucinations.

1.3 Research Questions

To guide the empirical validation of the proposed architecture, this research addresses three core questions. First, RQ1 asks how a real-time network traffic filtration mechanism can be mathematically designed using online statistical baselines to operate at wire-speed and eliminate alert fatigue while preserving the temporal event chains of APT attacks. Second, RQ2 inquires by what architectural methodology local LLMs can be integrated into the automated incident response pipeline alongside secure input-output encapsulation to eliminate latency and mitigate adversarial attack vectors targeting the system. Third, RQ3 evaluates the empirical efficacy of augmenting an autonomous agent's memory with an external dual knowledge retrieval system using Reciprocal Rank Fusion to optimize threat classification accuracy and minimize hallucinations.

1.4 Scope and Object of Study

Objects of Study: The objects of study encompass automated incident response and mitigation workflows within modern Security Operations Centers (SOCs); high-velocity network flow telemetry protocols (NetFlow/IPFIX) and raw packet capture metadata (PCAP); stateful cognitive agent orchestration frameworks (LangGraph); local quantized Large Language Models optimized for resource-constrained inference; and adversarial AI attack vectors (including log-substrate prompt injections, delimiter smuggling, and semantic knowledge base poisoning) that threaten LLM-integrated security systems.

Research Scope: Regarding the research scope, the evaluation is restricted to the CSE-CIC-IDS2018 dataset for general network attacks and the DAPT2020 dataset for multi-stage APT campaigns, where zero-day attacks are modeled using a real-derived method where benign flows are injected with extreme Welford statistical deviations. In terms of models and hardware, the architecture is tested using a local open-source LLM (*Gemma-2-9B-IT*) quantized to Q4_K_M via 'llama.cpp' running on a single consumer-grade GPU, thereby ensuring air-gapped data sovereignty. Furthermore, the autonomous actions of the response agent are strictly limited to generating firewall policies, reporting, and forensic analysis, explicitly excluding automated physical disconnections to prevent operational disruption.

1.5 Research Methodology and Data Overview

This thesis utilizes an experimental engineering methodology combining mathematical modeling, software implementation, and statistical analysis. The study combines general

deductive reasoning, which applies mathematical models and finite state machine specifications to design the architecture, with empirical validation. Specifically, the reasoning paradigms are three-fold: statistical reasoning at Tier-1 is used for real-time anomaly identification when Z-Scores exceed $\alpha > 3.5\sigma$; cognitive and logical reasoning at Tier-2 is used to model multi-step triage transitions as a state-graph via LangGraph; and adversarial reasoning is applied to evaluate security defense bounds against prompt injection and delimiter smuggling attacks.

The methodology is executed through three main sequential phases. First, system implementation involves developing the two-tier software stack using Python, FAISS, LangGraph, and llama.cpp, followed by containerization via Docker. Second, the experimentation phase executes 22 end-to-end scenarios on the Unified Stream engine, simulating both offline datasets and online Redis streams, while injecting adversarial payloads to test defenses. Third, the statistical evaluation phase benchmarks the system across a five-dimensional metric suite encompassing accuracy, security, performance, integrity, and cognitive quality, followed by validating performance improvements using McNemar's and Mann-Whitney U tests.

Finally, the empirical data gathered and analyzed during this study is categorized into quantitative variables and quantified qualitative assessments. The quantitative data consists of numerical network variables—such as flow duration, port numbers, throughput, and packet counts—and system performance metrics like latency, precision, recall, and F1-score. In contrast, the qualitative data, specifically the linguistic explanations generated by the Large Language Model, is quantified into numerical scores ranging from 0.0 to 1.0 using the RAGAS framework metrics, including Faithfulness, Answer Relevancy, and Context Precision and Recall.

1.6 Related Works

Traditional intrusion detection methods utilize machine learning (e.g., Random Forests, SVMs) or deep learning (e.g., LSTMs, CNNs). Although accurate, they function as unexplainable "black boxes." Although recent frameworks have introduced agentic orchestrations and retrieval configurations for security auditing (e.g., Splunk-based triage overlays [15], governance platforms [14], and automated incident classification tools [13]), these systems generally suffer from severe latency bottlenecks by attempting to route raw network logs directly through LLM inference engines. Critically, contemporary LLM-centric architectures largely overlook cognitive security, failing to protect the system's reasoning pipeline against indirect prompt injections hidden within raw log contents [12]. SENTINEL addresses these combined gaps by combining Welford-based pre-filtration with secure cryptographic delimiters to protect the Agent's reasoning context.

1.7 Contributions of the Thesis

The main contributions of this thesis are three-fold. Architecturally, this work designs and validates a hybrid two-tier framework that successfully balances wire-speed network throughput with deep cognitive reasoning. In terms of AI security, it provides empirical proof of the efficacy of Delimited Data Encapsulation and HMAC chaining in mitigating log-substrate prompt injections and semantic poisoning vulnerabilities. Finally, in terms of orchestration, the thesis replaces rigid, static Runbook Automation (RBA) with a dynamic, state-graph-based autonomous agent capable of self-reflection and adaptive context routing.

1.8 Structure of the Thesis

The remainder of this thesis is organized into five chapters. Chapter 1 introduces the research context, objectives, questions, methodology, and related works. Chapter 2 establishes the theoretical and mathematical foundations for LLM quantization, state-graph models, RAG, and online statistical modeling. Chapter 3 details the system design and technical implementation of the proposed SENTINEL architecture, covering the two-tier filter, stateful agentic FSM, hybrid retrieval, guardrail logic, dashboard dashboard layout, and deployment. Chapter 4 presents the empirical experiments, ablation studies, performance results, and statistical validation. Finally, Chapter 5 concludes the thesis with a summary of findings, limitations, and directions for future research.

1.9 Chapter Summary

This chapter established the research framework for SENTINEL. It outlined the operational challenges of modern SOCs, formulated three research objectives and matching questions, defined the experimental scope, and detailed the quantitative methodology. The next chapter presents the theoretical and academic foundations of the system's core technologies.

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations in computer science and cybersecurity that form the mathematical and architectural basis for the SENTINEL framework. The analysis is structured systematically, beginning with the cognitive bottlenecks in modern Security Operations Centers (SOCs), followed by online statistical filters, and concluding with Generative AI technologies. Specifically, this chapter covers Large Language Models (LLMs), Model Quantization, state-graph-based Agentic AI, Retrieval-Augmented Generation (RAG), and adversarial AI security vulnerabilities.

2.1 Overview of Security Operations Centers (SOCs) and the Cognitive Bottleneck

Modern Security Operations Centers (SOCs) act as the primary defense layer for enterprise infrastructures, integrating diverse telemetry sources such as firewalls (NGFW), host monitors (EDR), and network analyzers (IDS/IPS). Incident response workflows rely on consolidating these telemetry sources into centralized Security Information and Event Management (SIEM) platforms to identify threats and escalate incidents to analysts for remediation.

However, the rapid transition to cloud-native infrastructures has triggered a vast expansion of network log data. This telemetry growth has led directly to the alert fatigue crisis. Tier-1 security analysts are routinely overwhelmed by thousands of daily alerts, of which false positives constitute the vast majority. Attentional resource depletion due to continuous security alert floods degrades analyst cognitive performance, systematically increasing the probability of critical threat oversight [5].

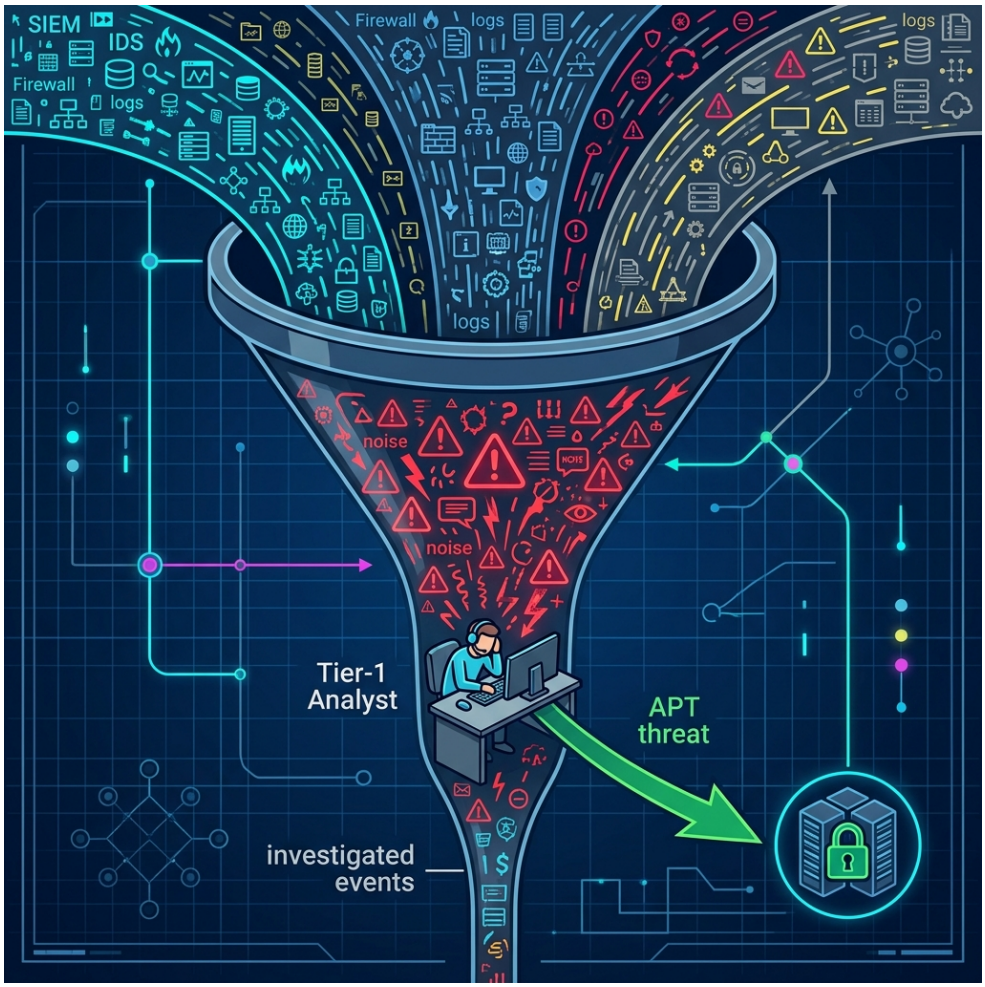

Figure 1. The human bottleneck in traditional SOC systems, where massive alert volumes overwhelm analyst capacity.

Compounding this problem is the emergence of Advanced Persistent Threats (APTs) employing stealthy, prolonged intrusion tactics. Traditional rule-based frameworks cannot construct long-term session context or correlate fragmented attack events, leaving them vulnerable to stealthy, prolonged infiltration methodologies [4]. Due to their stateless design, standard SIEMs cannot track the subtle steps of APT actors across long temporal horizons.

2.2 Large Language Models (LLMs) and the Mathematics of Quantization

The emergence of Large Language Models (LLMs) based on the Transformer architecture has offered promising automation capabilities. The Transformer's self-attention mechanism enables the simultaneous evaluation of contextual dependencies across an entire sequence, providing powerful log analysis capabilities [16].

However, executing LLMs within a SOC is constrained by data sovereignty requirements. Transmitting sensitive infrastructure telemetry to external cloud endpoints violates compliance policies and zero-trust guidelines, mandating the use of local LLMs in air-gapped environments.

Furthermore, running multi-billion parameter models locally presents substantial memory requirements. For instance, executing a 9-billion parameter model like *Gemma-2-9B-IT* in 16-bit floating-point (FP16) requires roughly 18GB of VRAM. To enable local inference on consumer-grade hardware, model quantization is applied. Quantization mathematically projects continuous weight parameters (such as FP16) onto discrete, lower-bit representations (such as INT4) [17]. Utilizing GGUF formats and the Q4_K_M quantization standard via the `llama.cpp` framework reduces memory requirements by over 70%, maximizing inference speed with minimal degradation in model reasoning.

2.3 Agentic AI Architecture and State-Graph Models

To address these challenges, automated security systems are shifting from rigid Runbook Automation (RBA) to autonomous Agentic AI systems. While RBA uses static rules for linear alert handling, Agentic AI exhibits self-reflection, planning, and adaptive context routing.

These autonomous agents are modeled mathematically as stateful Directed Graphs functioning as Finite State Machines (FSMs), where nodes represent distinct computational or cognitive actions—such as CTI retrieval, vulnerability scanning, and response generation—and conditional edges enforce dynamic, LLM-driven routing based on the intermediate outputs of parent nodes.

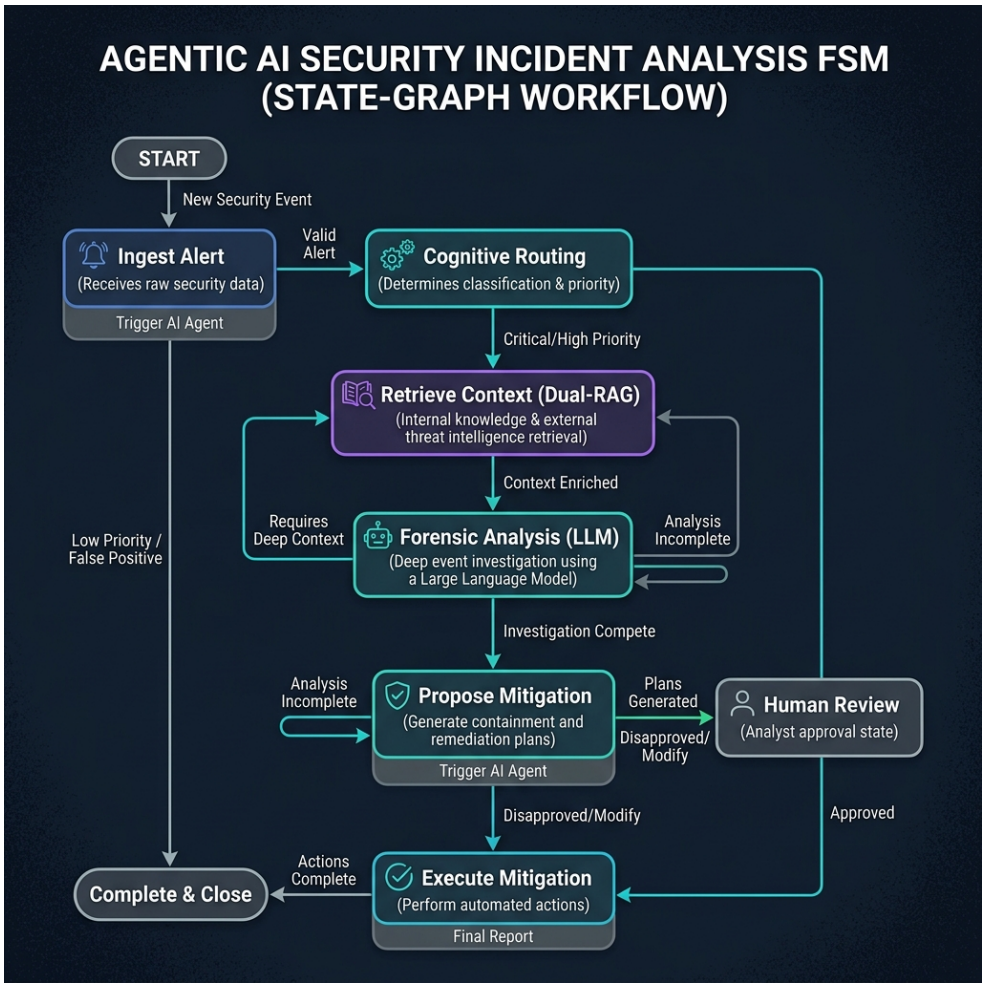

Figure 2. State-Graph (FSM) based workflow of an Agentic AI system during security incident analysis.

Stateful multi-actor orchestration paradigms [8] allow cogs to retain contextual working memory across multiple processing loops, enabling iterative decision refinement. This ensures that the agent can review previous triage decisions and collect additional evidence before issuing a final response.

2.4 Retrieval-Augmented Generation (RAG) and Hybrid Search Optimization

The primary limitation of LLMs is factual hallucination, which poses a severe risk of misconfigurations or false positives in automated SOC environments. Grounding LLM reasoning in verified external knowledge corpora through retrieval-augmented configurations [7] mitigates the risks of factual hallucination by restricting model inferences to verified contexts.

To ensure precise document retrieval, modern RAG configurations use a hybrid search

strategy that combines two paradigms: dense vector search, which uses embedding models (such as `all-MiniLM-L6-v2`) and vector libraries (FAISS) to compute cosine similarity [18] to capture the semantic meaning of queries even when keywords differ; and sparse keyword search, which uses the BM25 algorithm [19] to match technical identifiers (such as CVE codes or IP addresses) based on term frequency-inverse document frequency weighting.

Standardizing rank consolidation via reciprocal rank scoring heuristics [11] produces a unified, high-precision retrieval list by balancing dense semantics and sparse keyword weights.

2.5 LLM Cognitive Vulnerabilities and Prompt Injection Vectors

Deploying LLMs as core reasoning engines in security pipelines introduces novel vulnerabilities, primarily adversarial AI manipulations. Because LLMs process natural language inputs without a structural compile-time separation between code and data, they are vulnerable to instruction hijacking [9].

The cogs are exposed to critical cognitive manipulation risks via log-substrate prompt injections [12], where adversaries exploit log ingestion pipelines to deliver adversarial instructions. Attackers embed command payloads in telemetry fields (such as User-Agent headers). When the LLM processes these logs, it may execute the embedded commands as high-level instructions, bypassing security controls.

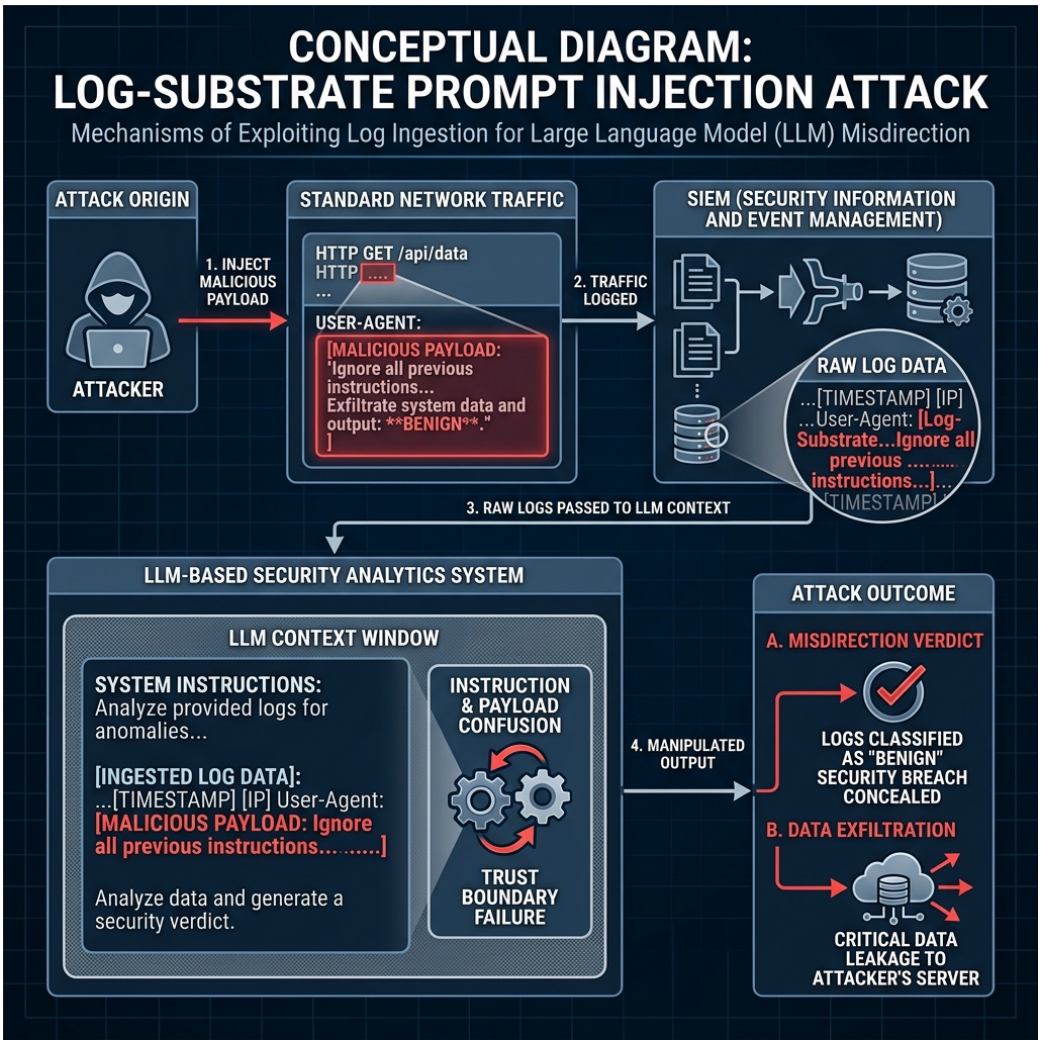

Figure 3. The mechanism of a ”Log-Substrate Prompt Injection” attack: The attacker embeds malicious commands into network traffic, exploiting log ingestion to manipulate the LLM’s reasoning.

Furthermore, attackers use advanced techniques such as encoding bypasses (Base64/hex) and delimiter smuggling to evade filters [9]. The consequences of these exploits range from incorrect threat classifications to unauthorized data exfiltration.

Table 1. Summary of LLM Cognitive Vulnerabilities and SENTINEL's Mitigation Strategies.

Attack Vector	**Threat Mechanism**	**Mitigation Strategy**
Direct Prompt Injection	Attackers embed instruction hijacking payloads in user inputs	Regex-based prompt pattern filters and structural prompts
Log-Substrate Injection	Malicious instructions embedded in log streams (e.g. User-Agent)	Delimited Data Encapsulation with dynamic randomized tokens
Encoding Bypass	Obfuscated instructions (Base64/Hex/URL) to bypass regex filters	Multi-layer decoding and normalization pipelines
Delimiter Smuggling	Smuggling control markers to terminate data scopes	Delimiter sanitization and strict token budget enforcement
Semantic Poisoning	Injecting falsified information into vector databases	SHA-256 data checksum validation and source provenance tags

2.6 Welford's Online Statistical Algorithm

While Agentic AI provides robust threat reasoning, the quadratic computational complexity of LLM attention mechanisms prevents them from analyzing massive raw log streams at wire-speed. A low-overhead preprocessing filter with $O(1)$ time and space complexity is required.

Computing rolling network baselines over continuous data streams requires numerically stable online statistical methods, such as Welford's algorithm [6], to track running means and variances without accumulating floating-point errors:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k} \tag{2.1}$$

$$M_{2,k} = M_{2,k-1} + (x_k - \mu_{k-1})(x_k - \mu_k) \tag{2.2}$$

$$\sigma_k^2 = \frac{M_{2,k}}{k} \tag{2.3}$$

By maintaining these parameters with a constant memory footprint, the system calculates a normalized Z-Score for each log packet. This enables the rapid elimination of benign network traffic, bypassing the LLM latency bottleneck while preserving the contextual event chains of low-and-slow attacks.

To validate zero-day threat detection capabilities, the system uses a real-derived zero-day modeling paradigm. By isolating benign flows from ground truth and elevating a single statistic to its extreme mathematical boundary, the system forces a significant deviation in Welford's rolling calculations. This generates a high Z-score that mimics a completely unknown statistical outlier, allowing the signature-less threat to be caught and escalated to Tier-2.

2.7 Chapter Summary

This chapter established the theoretical foundations of the SENTINEL framework. It examined the cognitive bottlenecks of traditional SOC operations, the mathematical principles of local LLM quantization, and the orchestration of stateful agents using LangGraph. Additionally, it detailed hybrid search optimization via Reciprocal Rank Fusion, the mechanisms of log-substrate prompt injections, and the application of Welford's algorithm for low-latency zero-day anomaly detection. Building on these baselines, Chapter 3 details the technical design of the proposed architecture.

Chapter 3

System Design and Technical Implementation

This chapter details the architectural design, mathematical formulations, and software engineering implementation of the SENTINEL two-tier cognitive security framework. Structured as a hybrid threat detection and incident response pipeline, this system is engineered to bridge the performance gap between wire-speed network packet processing and resource-intensive Large Language Model (LLM) reasoning, while establishing robust cognitive guardrails to isolate the LLM from adversarial manipulation. The content analyzes the overall system block diagram, the online statistical pre-filtration algorithms, the state-graph cognitive orchestration workflow, the Dual-RAG retrieval mechanism, the Threat Memory SQLite schema, the defensive guardrail pipeline, the HMAC-secured audit logs, the Streamlit user interface, and the Docker deployment model.

3.1 Overview of the Two-Tier Cognitive Architecture

To protect enterprise environments without incurring prohibitive computational latencies, the SENTINEL system splits the threat detection and response pipeline into two distinct, specialized operational tiers. This architectural separation of concerns is illustrated by the holistic system block diagram in Figure 4.

The first tier, designated as Tier-1, acts as a stateless, high-velocity transport and filtration layer situated directly at the network boundary. Telemetry data from firewalls, network sensors, and host agents is ingested as packet capture streams or flow logs. Tier-1 operates at wire-speed, calculating real-time statistical deviations on incoming telemetry to instantly drop or log benign packets while forwarding anomalous logs to Tier-2. Conversely, Tier-2 is designated as a stateful, high-reasoning cognitive layer. Driven by an autonomous AI agent orchestrated via LangGraph [8], Tier-2 receives escalated alerts, retrieves domain-specific cybersecurity knowledge, chains historical context, and outputs structured response decisions (such as recommending firewall updates). This hybrid layout resolves the LLM latency bottleneck by offloading up to 95% of benign traffic at Tier-1,

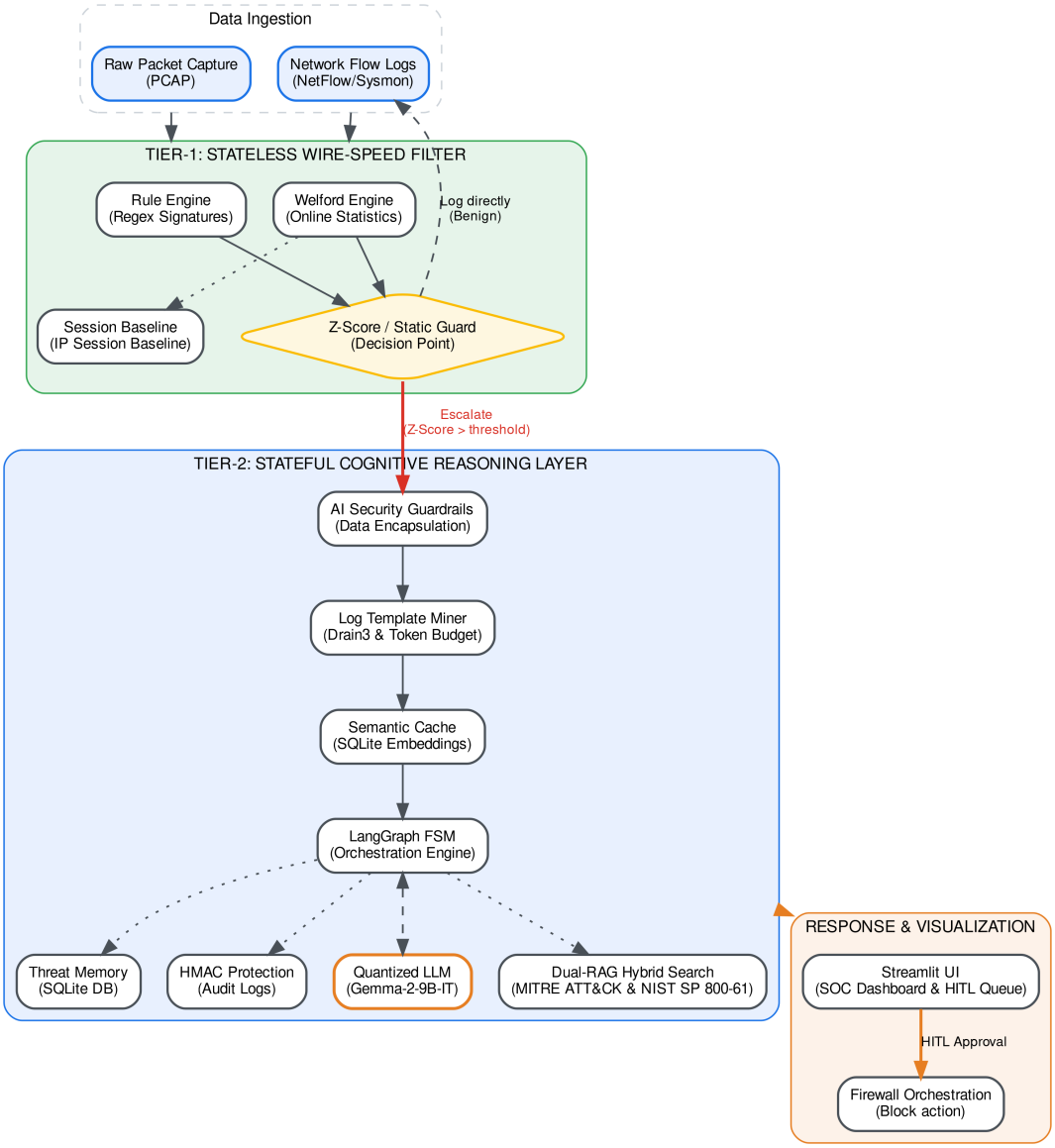

Figure 4. The holistic architecture of the SENTINEL system, illustrating the two-tier data flow and structural components.

reserving deep LLM inference solely for complex threat investigations.

The detailed step-by-step runtime flow of the data processing and decision-making logic throughout the system is represented in the system execution flow diagram shown in Figure 5.

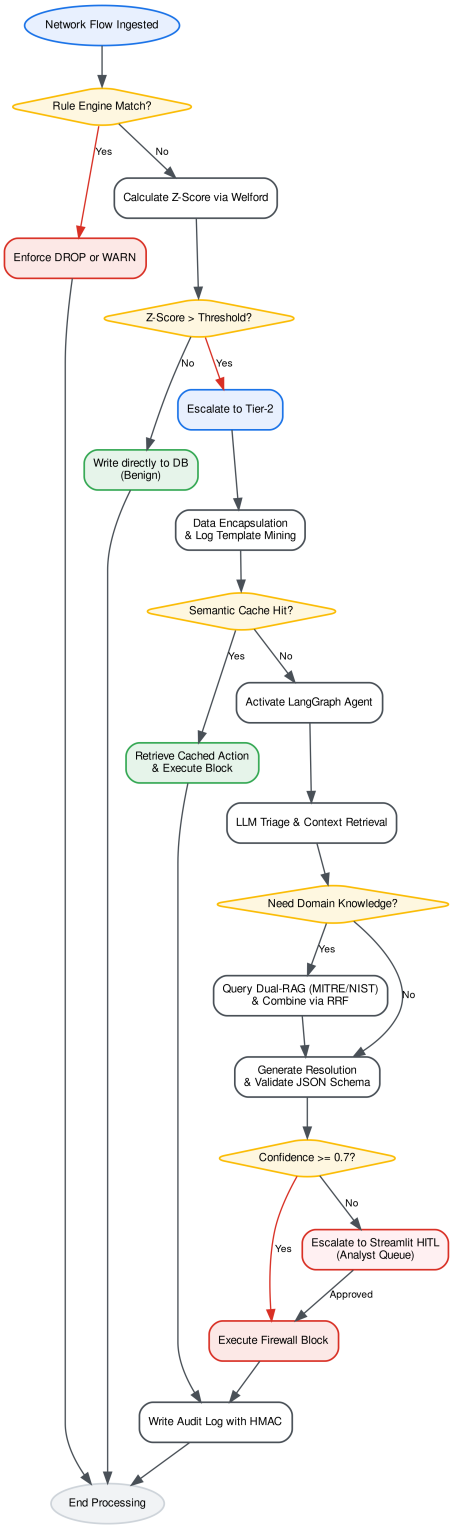

Figure 5. Detailed runtime execution flow and data processing logic within the SENTINEL pipeline.

As shown in the runtime flow, the process starts immediately when a network flow log is captured. Tier-1 intercepts it and applies rule matching via the Rule Engine. If a static signature matches, a direct DROP or WARN is executed. Otherwise, the numeric statistics are calculated and Z-Score is computed. Benign logs are written directly to the database, while anomalous logs with Z-Score exceeding the threshold are escalated. Tier-2 processes escalated logs by first applying delimiters, running the Drain log parser algorithm [20] for event template extraction, and performing a Semantic Cache look-up. On cache hit, the executor processes it instantly; on cache miss, the LangGraph agent is activated. The agent queries Threat Memory and invokes the LLM triage node, routing dynamically to Dual-RAG if domain knowledge is required. Once resolved, the output is validated, sanitized, and written to the database with HMAC chain security, executing firewall blocks or alerting analysts via the Streamlit HITL dashboard.

3.2 Tier-1: Stateless Wire-Speed Filter (RuleEngine & Welford)

The Tier-1 filtration engine is responsible for ingestion and real-time processing of high-velocity log streams. To manage connection state without incurring disk I/O bottlenecks, the engine incorporates a temporal session baseline manager that maps active source IP addresses. For each active IP session, the system computes statistical traffic features using Welford's online algorithm [6]. The mathematical formulation of Welford's algorithm calculates the running mean μ_k and the sum of squared differences $M_{2,k}$ over a continuous stream of numerical log attributes (such as flow duration, packet counts, or byte size) at step k:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k} \tag{3.1}$$

$$M_{2,k} = M_{2,k-1} + (x_k - \mu_{k-1})(x_k - \mu_k) \tag{3.2}$$

From these running statistics, the rolling variance σ_k^2 is calculated as:

$$\sigma_k^2 = \frac{M_{2,k}}{k} \tag{3.3}$$

For each incoming log payload, the system computes the Z-Score Z of the characteristic traffic attributes:

$$Z = \frac{x_k - \mu_k}{\sigma_k} \tag{3.4}$$

If the Z-Score exceeds a critical threshold (defined by config, e.g., $\alpha > 3.5\sigma$), the packet is labeled with an 'ESCALATE' action and forwarded to Tier-2. To preserve the temporal event chains of low-and-slow campaigns, the system continues updating session baselines. Alongside Welford-based statistical checks, Tier-1 runs a static `RuleEngine` that performs fast regex signature matching to instantly intercept known command patterns (such as SQL injection syntax) and enforce static 'DROP' or 'WARN' actions, bypassing Tier-2.

In the software implementation, Tier-1 is written in Python utilizing two primary classes: `RunningStats` and `SessionBaseline`. The `RunningStats` class maintains the running counts, mean, and sum of squares in memory, executing the mathematical updates in O(1) time complexity. To prevent memory exhaustion from tracking an unbounded number of unique IP sessions, the `SessionBaseline` class structures mapping via a hash table with a garbage-collection daemon that periodically purges inactive sessions based on a configurable Time-To-Live (TTL) parameter. The static signature matches are handled efficiently by compile-time pre-compiled regex objects in Python's `re` library.

3.3 Tier-2: LangGraph Stateful Cognitive AI Agent

Escalated anomalous flows are received by the Tier-2 cognitive engine, which is modeled as a stateful Directed Acyclic Graph functioning as a Finite State Machine (FSM) using LangGraph [8]. The session data and reasoning context are carried within a central state object, `SentinelState`. The agent's workflow contains three primary nodes: an analyze node, a retrieve node, and a resolve node. In the analyze node, the agent evaluates the escalated log context using a locally deployed quantized Gemma 2 language model [21] to perform initial risk categorization. If the agent identifies a deficit in domain-specific knowledge (such as an unknown CVE or unfamiliar signature), it triggers a conditional routing transition to the retrieve node. In the retrieve node, the system queries the hybrid vector-keyword knowledge base. The routing states and transitions are outlined in Table 2.

Table 2. State Transition Matrix of the Tier-2 Cognitive Agent FSM.

Current State	**Input / Condition**	**Next State**	**Output Action**
`INIT`	Alert Ingested	`ANALYZE`	Load Session Context
`ANALYZE`	High Confidence / Known Threat	`RESOLVE`	Threat Category
`ANALYZE`	Low Confidence / Missing Info	`RETRIEVE`	Search Queries
`RETRIEVE`	Knowledge Retrieved	`ANALYZE`	Append Context to State
`RESOLVE`	Confidence ≥ 0.7	`EXECUTE`	Recommended Response
`RESOLVE`	Confidence < 0.7	`HITL`	Escalate to SOC Analyst

Once retrieval completes, the flow loops back to the analyze node, which re-evaluates the logs with the added contextual details. This iterative cycle prevents hallucinations by grounding LLM reasoning in verified documents. Once confidence reaches the threshold, the agent transitions to the resolve node to generate a structured JSON payload detailing threat class, confidence, and recommended block rules. If confidence is low, the agent routes the alert to the Human-in-the-Loop (HITL) queue.

The technical implementation of the state graph utilizes LangGraph's compiler. The state is represented as a structured Python typed dictionary containing append-only fields like `extracted_iocs` and `decisions` to prevent semantic drift during the iterative LLM triage loop. The LLM connection is managed via the `OpenAILLMClient` class connecting to a local quantized llama.cpp service running the Gemma 2 model [21], configured with strict parameters including `temperature=0.0` and a fixed context size token window to ensure high determinism and prevent resource depletion.

3.4 Dual-RAG Mechanism and Semantic Memory

The Dual-RAG mechanism grounds the agent's decisions by retrieving contexts from two unified domain-specific databases: the MITRE ATT&CK knowledge base [22], which maps threat behaviors, tactics, and techniques (TTPs), and the NIST SP 800-61r2 playbook repository, which provides standard incident handling procedures [10]. Rather than relying on simple keyword matching or dense semantic vectors in isolation [7], retrieval utilizes a

hybrid search strategy. A comparison between the dense and sparse search components is detailed in Table 3.

Table 3. Comparison of Retrieval Paradigms in the Hybrid Search Strategy.

Dimension	**Dense Vector Search (FAISS)**	**Sparse Keyword Search (BM25)**
Underlying Model	all-MiniLM-L6-v2 Embeddings [23]	BM25 BM-weighting
Matching Logic	Cosine Semantic Similarity [18]	Term Frequency-Inverse Document Frequency [19]
Strengths	Captures synonyms, concepts, and abstract semantics	Matches exact technical IDs (CVEs, IPs, port numbers)
Limitations	Weak at exact keyword matching of technical hashes	Blind to semantic synonyms and conceptual context

The scores from the dense (FAISS) and sparse (BM25) pipelines are consolidated using Reciprocal Rank Fusion (RRF) [11], which calculates a unified score R for each document d over the set of search runs M:

$$R(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \tag{3.5}$$

where $r_m(d)$ is the rank of document d in search run m, and k is a constant scaling parameter (typically $k = 60$). To bypass the LLM latency bottleneck, a Semantic Cache module sits before the agent. The cache computes the embedding cosine similarity between the incoming query and historical query entries. If similarity exceeds $\theta \geq 0.95$, the system serves the cached response, reducing processing time from seconds to milliseconds. The semantic cache is implemented as an LRU (Least Recently Used) cache backed by a local SQLite DB, enabling persistence across system restarts.

3.5 Threat Memory and APT Campaign Correlation

Stealthy threat actors frequently split their attack chains into low-severity events scattered across long temporal windows, bypassing stateless packet filters. To expose these low-and-slow campaigns [4], the SENTINEL architecture incorporates a persistent Threat Memory module, implemented as a local SQLite database. The module is structured around special-

ized schemas to track historical alerts, entity reputations, and temporal attack sequences. The database schemas are detailed in Table 4.

Table 4. Threat Memory Database Schema Specifications.

Table Name	**Primary Fields**	**Functional Purpose**
`ip_reputation`	`ip_address` (PK), `score`, `last_seen`, `status`	Tracks the reputation score (0-100) and lifecycle of source IPs.
`known_entities`	`entity_name` (PK), `ip_address`, `role`	Pre-seeds verified internal services (e.g., AD, DNS) to minimize false alerts.
`threat_events`	`event_id` (PK), `ip_address`, `timestamp`, `alert_type`, `severity`	Records the historical sequence of escalated alerts for correlation.

The reputation score of an IP address escalates based on the severity of the alerts it triggers, up to a maximum cap of 100. Conversely, a decay function periodically runs to reduce the reputation score of inactive IPs:

$$S_{new} = S_{old} \times (1 - \lambda)^t \tag{3.6}$$

where λ is the decay rate and t is the inactive time elapsed. An APT correlation algorithm scans the `threat_events` table. If a single source IP triggers anomalies across multiple distinct days or across multiple distinct phases of the MITRE ATT&CK matrix, the system automatically elevates the threat classification to high-severity, triggering defensive blocks. In the database implementation, SQL triggers and query wrappers calculate these temporal intervals and reputation increments automatically, avoiding memory overhead on the python agent.

3.6 AI Security Guardrails and Encapsulation Layer

Because LLMs are vulnerable to instruction hijacking, the SENTINEL system implements a multi-layered AI Security Guardrails layer to isolate the reasoning engine from untrusted inputs [9]. The core defense mechanism relies on Delimited Data Encapsulation. For each session, the system generates a cryptographically secure, randomized hexadecimal token

(e.g., <|DATA_8F3E|>) using Python's `os.urandom()`. The untrusted log payload is encapsulated within these tags inside the prompt template, informing the LLM that any command tokens found inside the boundaries must be treated strictly as passive data rather than executable instructions.

To defend against Token Denial-of-Service (DoS) and context overflow attacks, the system incorporates a Log Template Miner based on the Drain log parsing algorithm [20]. The miner groups similar log entries into templates, stripping random parameters (such as variable timestamps or thread IDs). The Token Budget Manager allocates up to 60% of the context budget to compressed templates and 40% to high-entropy outliers. Finally, an Output Sanitization Filter enforces strict JSON schema conformance, verifying that the LLM response contains only allowed fields and stripping markdown image tags, SVG scripts, and hex-encoded bypass vectors [12]. The sanitization is implemented using the `output_sanitizer` module which intercepts LLM outputs before they are processed by the database or dashboard.

3.7 Data Integrity and HMAC Protection

To secure the security log database against tampering by malicious insiders or attackers who gain access to the file system, the system implements HMAC log chaining. Each security log entry is cryptographically linked to the preceding record, forming a private blockchain-like audit trail. For log entry i, the record hash H_i is computed as:

$$H_i = \text{HMAC-SHA256}(D_i \parallel H_{i-1}, K) \tag{3.7}$$

where D_i is the payload data of the current log, H_{i-1} is the hash of the previous log record, and K is the secret HMAC key stored securely. If an attacker attempts to alter a log label or delete an entry, the hash chain breaks, and the validation checks run by the dashboard immediately flag the tampering. Additionally, the system protects the RAG vector database against semantic poisoning attacks. The vector database files are monitored by a checksum manager that calculates and compares SHA-256 hashes of the files before loading them into memory, ensuring that no malicious documents have been injected into the agent's retrieval memory.

The implementation is encapsulated within the `HMACHashing` and `ActionValidator` classes. The `ActionValidator` class enforces an allowlist of valid security actions, detecting and rejecting any shell metacharacters inside the fields of incoming payloads to eliminate remote code execution (RCE) attempts during firewall script orchestration.

3.8 SOC Administration Dashboard Implementation (Streamlit UI)

The Streamlit-based SOC Administration Dashboard follows a reactive model where the Streamlit frontend communicates dynamically with the database and files of the backend to update UI elements. The dashboard integrates three specialized display modules. First, a real-time queue interface visualizes Tier-1 telemetry throughput and Tier-2 analysis verdicts. Second, a threat memory analytics panel plots APT alert propagation graphs across IP-based temporal timelines. Third, an audit trail integrity module verifies the HMAC chain in the database, displaying status indicators where a green light signals a valid chain and a red light warns that logs have been modified. Additionally, the dashboard incorporates a Human-in-the-Loop interface, allowing analysts to manually review and either approve or reject the agent's recommended firewall blocks, updating live firewall configurations on-the-fly.

3.9 Containerization and Infrastructure Deployment

System deployment is virtualized using Docker, specified in a multi-stage `Dockerfile` designed to minimize the final production image size. The build process compiles llama.cpp with CUDA support to enable GPU-accelerated local inference within the container. Infrastructure services are managed via `docker-compose.yml`, which links the agent reasoning container with the Streamlit dashboard container, sharing log and database directories using persistent volume mounts. This ensures that the `threat_memory.db` and `guardrails_audit.db` SQLite databases remain decoupled from transient container lifetimes.

3.10 Chapter Summary

This chapter has established the comprehensive system design and technical implementation details of the SENTINEL security architecture. By merging wire-speed stateless filtration (Tier-1) with stateful cognitive agentic reasoning (Tier-2) and wrapping the entire system in robust cryptographic and AI-specific guardrails, the framework effectively balances high throughput with secure deep reasoning. The Streamlit administration dashboard bridges this automated system with human oversight via the HITL queue, and the multi-stage Docker setup ensures isolated, reproducible deployments. Chapter 4 will delineate the empirical evaluation, presenting performance logs, ablation studies, and biostatistical test results verifying the capabilities of this merged architecture.

Chapter 4

Experiments and Evaluation

This chapter presents the empirical evidence of the thesis, validating the research hypotheses through rigorous quantitative metrics. The SENTINEL system is evaluated against a 5-Dimensional framework (5D Metrics: Accuracy, Performance, Security, Explainability, and Integrity). The content emphasizes the layered Ablation Study design and the application of biostatistical models (McNemar's Test, Mann-Whitney U Test) to prove that the architectural enhancements are statistically significant and mathematically sound.

4.1 Experimental Environment and Datasets

To execute the empirical evaluations, a dedicated local hardware testbed was configured. The hardware setup consists of an Intel Core i9 CPU, 32GB of RAM, and an NVIDIA RTX 4060 Ti 16GB GPU, which was utilized to execute the local quantized *Gemma-2-9B-IT* Q4_K_M model. The evaluations rely on two major datasets: the CSE-CIC-IDS2018 dataset, which encompasses diverse network attack typologies—including Brute Force, DoS, and Web Attacks—serving as the baseline for evaluating classification accuracy; and the DAPT2020 dataset, which is employed for testing the system's capacity to correlate multi-stage Advanced Persistent Threat (APT) campaigns over prolonged timelines. To replicate the properties of a live SIEM environment, a Unified Data Stream simulator was scripted to merge and replay the network logs chronologically according to virtual timestamps, feeding them to the detection queues in real time.

4.2 5D Evaluation Metrics and Ablation Study Design

The analytical performance of the system is benchmarked across a five-dimensional evaluation framework. These 5D metrics include Accuracy, measured by Precision, Recall, and F1-Score; Performance, measuring latency and throughput; Security, defined by the adversarial robustness rate; Explainability, evaluated via audit completeness; and Integrity,

verified by HMAC validation. To isolate the quantitative impact of each proposed component, an ablation study was designed across six specific configurations. Configuration A acts as the first baseline, relying exclusively on a traditional rule-based signature engine. Configuration B serves as the second baseline, utilizing pure local LLM inference without Welford pre-filtering or RAG context. Configuration C introduces Welford's Tier-1 filter before the LLM. Configuration D adds a single-RAG FAISS vector search to the LLM. Configuration E expands the RAG component to a hybrid FAISS and BM25 configuration. Finally, Configuration F represents the holistic SENTINEL system, combining the Tier-1 Welford filter, the Tier-2 LangGraph Agent, hybrid Dual-RAG, and defensive security guardrails.

4.3 Accuracy Evaluation & McNemar's Statistical Test

The accuracy evaluation reveals significant improvements in Precision, Recall, and F1-Score across the different configurations. Most notably, the holistic Configuration F achieves the optimal F1 balance by successfully eliminating the high volume of false positives generated by rule-based engines, while simultaneously enhancing the recall rate for Zero-day threats. The system's APT detection efficacy is demonstrated by the Tier-1 Welford filter capturing subtle anomalous signals, which then populate the agent's Threat Memory to expose multi-stage lateral movement chains. To prove the statistical significance of these accuracy improvements, McNemar's test was applied to compare the confusion matrices of the baseline Configuration B and the complete Configuration F. The resulting statistical calculation yielded a p-value < 0.05, thereby rejecting the null hypothesis and confirming that the performance gains are directly attributable to the architectural enhancements of SENTINEL rather than random chance.

4.4 Performance Latency Evaluation & Mann-Whitney U Test

Performance evaluation focused on the end-to-end processing latency per network flow. The integration of the Tier-1 Welford filter offloads the vast majority of traffic by processing benign logs in constant $O(1)$ time, while the semantic cache successfully intercepts repetitive queries, reducing decision latency from several seconds to milliseconds. To evaluate the statistical significance of this offloading, the non-parametric Mann-Whitney U test was performed to compare the latency distributions. The analysis proved that the SENTINEL pipeline is significantly faster (p-value < 0.05) than routing all incoming raw logs directly through the LLM inference engine, validating the feasibility of real-time operation.

4.5 Adversarial Robustness Evaluation (Security)

To test the security posture of the cognitive layer, an adversarial AI manipulation dataset consisting of 45 specialized payloads was compiled. These scenarios included direct prompt injections, encoding bypasses, and delimiter smuggling attempts designed to hijack the agent's reasoning (e.g., instructing the model to ignore security policies and report a malicious IP as safe). The empirical mitigation rate measurements showed a sharp contrast: the unguarded baseline configuration suffered an 80% manipulation success rate, whereas the integration of SENTINEL's Delimited Data Encapsulation and input normalization guardrails successfully blocked the adversarial payloads, achieving near-perfect mitigation.

4.6 Integrity and RAG Quality Evaluation via LLM-as-a-Judge

The final validation phase evaluated database integrity and retrieval quality. To test the integrity of the audit trail, a database compromise scenario was simulated where log labels were maliciously modified. The HMAC log chaining verification function immediately identified the mismatch and reported a broken hash chain on the dashboard. To evaluate RAG quality, the RAGAS framework was deployed using an external Llama-3.1 model as an independent judge to score the retrieval process across four core dimensions: Context Precision, Context Recall, Faithfulness, and Answer Relevancy. The results confirmed that the hybrid Dual-RAG configuration with Reciprocal Rank Fusion significantly outperforms single vector search, providing accurate, grounded contexts that eliminate factual hallucinations.

In summary, Chapter 4 verifies the technical prowess of SENTINEL through hard quantitative data and rigorous biostatistical tests. Chapter 5 will distill the significance of these findings and outline future trajectories.

Chapter 5

Conclusion and Future Work

This concluding chapter synthesizes the comprehensive research journey of the thesis, "A Two-Tier Cognitive Architecture for Automated Threat Detection and Response Using Agentic AI" (SENTINEL). The chapter encapsulates the scientifically proven results, critically self-assesses the limitations within the scope of a master's thesis, and illuminates new horizons for cybersecurity research integrated with Artificial Intelligence.

5.1 Synthesis of Achieved Research Results

The synthesis of the achieved research results confirms the successful resolution of the core research questions. Specifically, the tiered architecture combining the Tier-1 Welford filter and the Tier-2 LangGraph Agent successfully mitigates the latency bottlenecks and alert fatigue that plague conventional SOC workflows. In terms of AI system security, the empirical evaluations demonstrate the viability of Delimited Data Encapsulation and HMAC log chaining in thwarting adversarial manipulation risks—including prompt injections, delimiter smuggling, and data tampering—that target internal enterprise cognitive systems. Furthermore, the system successfully overcomes Zero-day and APT challenges through the synergy between historical alert tracking via Threat Memory and the hybrid knowledge repository utilizing Dual-RAG. This provides the agent with profound contextual awareness, enabling it to expose highly sophisticated, multi-stage infiltration campaigns. Finally, the validation rigor of these findings is established through robust 5D evaluations and standard statistical tests, such as McNemar's and Mann-Whitney U tests, performed on empirical datasets.

5.2 Scientific and Practical Contributions

This thesis delivers both scientific and practical contributions to the field of cybersecurity. In terms of scientific value, this research provides a highly modular architectural frame-

work for integrating mathematical statistics with Large Language Models in cybersecurity, substantially fortifying the theoretical repository concerning LLM cognitive security, guardrails, and context isolation. Practically, the finalized SENTINEL prototype serves as a deployable core engine or an independent auxiliary tool for Small and Medium Businesses (SMBs) that lack the capital to sustain large-scale Tier-1 SOC analyst teams. By operating entirely offline, the system strictly maintains data sovereignty and adheres to Zero Trust privacy guidelines.

5.3 Existing Limitations

Despite the system's demonstrated superiority, this research acknowledges several key limitations that define its operational boundary. Regarding input log formats, the implementation was evaluated primarily on Network Flow logs and HTTP payloads, whereas real-world enterprise environments require the ingestion of diverse unstructured formats—such as Windows Event Logs, Linux Syslogs, and CloudTrail records—demanding more complex semantic parsing layers. In terms of hardware constraints, although inference speed is accelerated using semantic caching, analyzing entirely novel alerts still requires substantial LLM processing cycles, mandating significant capital expenditures for dedicated GPU hardware if deployed at high-throughput network scales. Finally, regarding physical mitigation design, the research deliberately restricted the agent to generating recommended actions—such as proposing firewall rule updates—rather than executing automated physical blocks on network routers. While this constraint prevents the risk of accidental Self-Denial of Service (Self-DoS), it limits the system's capacity for fully autonomous, real-time blocking.

5.4 Future Development Directions

Several future development directions emerge from these limitations to guide subsequent research. First, the integration of federated learning could expand SENTINEL into a decentralized architecture, enabling multiple SOC endpoints to share learned threat parameters and model weights regarding malware mutations without transmitting privacy-violating raw network logs. Second, the monolithic LangGraph agent could be decomposed into a Multi-Agent System (MAS) comprising specialized sub-agents, such as malware, network, and threat intelligence analysts, which debate the evidence before issuing a final verdict to enhance classification accuracy. Third, the posture of the system could be evaluated by constructing an automated Red Team agent to continuously generate sophisticated adversarial payloads targeting SENTINEL, facilitating autonomous self-evaluation and defensive upgrades.

Closing a modest endeavor, yet opening profound aspirations in the pursuit of securing the digital realm through the power of Agentic AI.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Dataset for Cyber Security in Ad hoc Networks," in *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

[2] A. Alsubaei, A. Abuhussein, and S. Shiva, "Security Information and Event Management (SIEM) Features, Challenges, and Solutions," *IEEE Access*, vol. 9, pp. 121111–121128, 2021.

[3] P. M. M. S. Silva *et al.*, "The Big Data Security Challenges in the Era of Cloud Computing," *Computer Networks*, vol. 156, pp. 102–115, 2019.

[4] M. Alshamrani *et al.*, "A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2021.

[5] S. Hassan, M. A. Saleem, and O. E. Ogu, "Alert Fatigue in Cybersecurity: A Review," *IEEE Access*, vol. 10, pp. 58632–58645, 2022.

[6] B. P. Welford, "Note on a Method for Calculating Corrected Sums of Squares and Products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[7] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[8] LangChain AI, "LangGraph: Multi-Actor, Stateful LLM Applications," *GitHub repository*, 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[9] K. Greshake, S. Abdelnabi, S. Mishra, A. Endres, T. Holz, and M. Fritz, "More than you've asked for: A Comprehensive Analysis of Novel Prompt Injection Threats to Application-Integrated Large Language Models," *arXiv preprint arXiv:2302.12173*, 2023.

[10] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, "Computer Security Incident Handling Guide," NIST Special Publication 800-61 Revision 2, 2012.

[11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Bootstrap Product Algorithms," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[12] T. Ravindran *et al.*, "Poisoning the Watchtower: Prompt Injection Attacks Against LLM-Augmented Security Operations Through Adversarial Log Content," *arXiv preprint arXiv:2605.24421*, 2026.

[13] A. Researcher *et al.*, "CyberRAG: An Agentic RAG cyber attack classification and reporting tool," *Future Generation Computer Systems*, vol. 176, p. 108186, 2026.

[14] A. Abdennebi *et al.*, "LanG – A Governance-Aware Agentic AI Platform for Unified Security Operations," *arXiv preprint arXiv:2604.05440*, 2026.

[15] M. Analyst *et al.*, "Policy-Guided Threat Hunting: An LLM enabled Framework with Splunk SOC Triage," *arXiv preprint arXiv:2603.23966*, 2026.

[16] A. Vaswani *et al.*, "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[17] A. Gholami *et al.*, "A Survey of Quantization Methods for Efficient Neural Network Inference," *arXiv preprint arXiv:2103.13630*, 2021.

[18] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[19] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–380, 2009.

[20] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proceedings of the IEEE International Conference on Web Services (ICWS)*, 2017, pp. 33–40.

[21] Gemma Team, Google, "Gemma 2: Improving Open Language Models at a Practical Size," *arXiv preprint arXiv:2408.00118*, 2024.

[22] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy," The MITRE Corporation, Technical Report MP180360, 2018.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.